Design and Implementation of Aurora V: A RISC-V SoC Featuring Fixed and Floating-Point Units with an Integrated FFT Coprocessor for DSP and Embedded Applications

Team: RTL Rangers

Bachelor of Engineering
School of Electronics and Communication Engineering
KLE Technological University, Hubballi

By:

1. **SHREEVATSA ALAWANDI**
2. **UMER YAKUB KUDACHI**
3. **AISHWARYA BEEDANAL**

Contents

1 Introduction **6**

2 Design Details **8**
2.1 Introduction . . . 8
2.1.1 Fetch Unit . . . 8
2.1.2 Decode Stage . . . 9
2.1.3 Execute Unit . . . 10
2.1.4 Floating Point Module . . . 10
2.1.5 Memory Access and Writeback . . . 11
2.1.6 Hazard Management System . . . 11
2.1.7 SPI Communication Module . . . 11
2.1.8 FFT Co-Processor . . . 12

3 Prototyping / Results **18**
3.0.1 Maximum Clock Frequency . . . 18
3.0.2 Power Consumption Report . . . 19
3.0.3 Resource Utilization . . . 20

4 Market Opportunity **21**
4.0.1 Embedded Systems . . . 21
4.0.2 Industrial IoT (IIoT) and Automation . . . 21
4.0.3 Digital Signal Processing (DSP) Applications . . . 22

5 Competitive landscape **25**
5.0.1 Overview . . . 25
5.0.2 Feature Comparison . . . 25
5.0.3 Market Positioning . . . 25

6 Budget Requirements **27**

7 Resource Mobilisation Plan **28**
7.0.1 Human Resources . . . 28
7.0.2 Hardware and Tools . . . 28
7.0.3 Software and IP Resources . . . 29
7.0.4 Strategic Partnerships . . . 29

8 Summary **30**

List of Tables

2.1 Supported Instructions in the RISC-V Processor 10
2.2 SPI Timing and Data Rate Parameters 12

3.1 Power Consumption Report of the Synthesized Processor 19

5.1 Core Overview of Compared Processors 25
5.2 Feature Comparison with Competitors 25

6.1 Expected Budget for Typical Processor Project 27
6.2 Actual Budget for Aurora-V Processor Development 27

7.1 Human Resource Allocation 28
7.2 Hardware Resources 28
7.3 Software and IP Resource Allocation 29
7.4 External and Institutional Support 29

List of Figures

2.1 Architecture of the Proposed RISC V Core. Readers are advised to zoom in to observe finer details. 8
2.2 Block Diagram of the SoC 14
2.3 RTL Schematic of the Floating Point ALU 15
2.4 Top RTL schematic of the processor 15
2.5 Worst Case Scenario of Hazard Unit 15
2.6 SPI communication with a slave 16
2.7 Simulation of 8 pt dit fft 16
2.8 Micro-architecture of the FFT Unit 17

3.1 Simulation of Area of Circle 18
3.2 Experimental setup showing communication with a slave; 'A' displayed on the seven-segment display via the processor 18
3.3 FPGA Resource Utilisation of RISC-V Processor Modules 20

4.1 Market Growth: Embedded Systems vs DSP (2023–2030) 23
4.2 IIoT Market Growth Forecast (2024–2032) 23
4.3 Software vs Hardware FFT Performance Comparison 24

5.1 Market Positioning: Flexibility vs. Performance 26

Chapter 1

Introduction

Concept

Aurora V is a custom RISC-V-based processor system featuring an integrated FFT coprocessor, floating-point unit (FPU), and a high-speed SPI interface, engineered for efficient real-time digital signal processing (DSP) tasks.
Aurora V is specifically designed to execute 8-point FFT operations on live sensor data—such as ECG or EEG signals—streamed through the SPI interface. The architecture uses a shared-memory approach with advanced synchronization logic, allowing the main processor and FFT coprocessor to work in harmony without delays or memory access conflicts.
This design makes Aurora V ideal for embedded applications requiring rapid and precise spectral analysis, such as biomedical monitoring, wearable health devices, and portable spectrum analyzers, providing low latency and high accuracy in edge computing environments.

Problem Statement and Benefits

Real-time processing of biomedical signals such as ECG and EEG is critical for timely diagnosis and monitoring of health conditions. However, conventional embedded processors often face challenges in efficiently performing computationally intensive tasks like Fast Fourier Transform (FFT), resulting in high latency and energy consumption. Additionally, slow or inefficient communication interfaces can bottleneck data acquisition from sensors, further limiting system responsiveness.
Aurora V addresses these challenges by integrating a dedicated FFT coprocessor and a high-speed SPI interface within a RISC-V processor architecture. The FFT coprocessor offloads spectral analysis tasks, significantly reducing processing time and enabling real-time frequency domain analysis. The high-speed SPI module ensures fast and reliable data streaming from external sensors, while the shared memory synchronization mechanism prevents memory access conflicts, minimizing delays.
This processor is needed to enable accurate, low-latency signal analysis in embedded and wearable medical devices, portable spectrum analyzers, and other edge computing applications where real-time insights are essential. Aurora V's architecture enhances performance and energy efficiency, making advanced signal processing feasible on resource-constrained platforms.

Novelty and Uniqueness

Aurora V offers several innovative features that differentiate it from existing processor solutions:

- **Dedicated FFT Coprocessor with Advanced Shared Memory Synchronization:** Traditional processors often execute FFT computations on the main CPU, leading to high processing latency and reduced efficiency. Aurora V integrates a specialized FFT coprocessor that offloads these intensive computations, enabling faster spectral analysis. The coprocessor shares memory with the main processor through a carefully designed synchronization mechanism that manages access without causing stalls or data corruption. This approach ensures smooth, concurrent operation, significantly improving throughput and reducing idle times compared to designs lacking coordinated memory sharing.
- **Floating-Point Unit for High-Precision FFT Computations:** Many embedded FFT implementations rely on fixed-point arithmetic, which can compromise accuracy, especially when analyzing low-amplitude biomedical signals like ECG and EEG. Aurora V's floating-point unit allows the FFT coprocessor to perform calculations with greater numerical precision, enhancing the reliability and quality of signal analysis in sensitive applications.
- **Optimized High-Speed SPI Interface for Real-Time Data Acquisition:** Fast and efficient communication with external sensors is crucial in embedded signal processing. Aurora V includes an optimized SPI module that supports high-throughput data streaming with minimal latency, ensuring that sensor data is promptly available for processing by the FFT coprocessor. This reduces bottlenecks commonly caused by slower peripheral interfaces.
- **Modular and Scalable Architecture:** The design philosophy of Aurora V emphasizes modularity, allowing individual components—the main processor, FFT coprocessor, and SPI module—to be developed, tested, and upgraded independently. This facilitates easier debugging, potential future integration of additional accelerators (e.g., filters, neural network engines), and customization for varied applications without redesigning the entire system.

Together, these features provide a highly efficient, precise, and flexible platform tailored for demanding embedded signal processing tasks, outperforming many traditional processor designs in both speed and accuracy.

Chapter 2

Design Details

2.1 Introduction

The section outlines the structural and functional blueprint of the processor. The processor was built using AMD's Vi- vado Design Suite version 2024.2 and follows a classic five- stage pipeline approach, as illustrated in Fig. 1. The pipeline includes the following sequential stages: (i) Fetching the in- struction, (ii) Decoding the fetched instruction, (iii) Executing the operation, (iv) Accessing memory , and (v) Writing the result back to the register file. The processor maintains an in- order execution model across all these stages. The detailed explanation of the processor's top-level architecture and its internal pipeline components is presented in the upcoming subsections.

2.1.1 Fetch Unit

The first stage of the processor pipeline is the fetch stage, where the primary function is to retrieve the next instruction to be executed. At the beginning of this stage, the Program Counter (PC) is incremented to point to the memory address of the upcoming instruction. This updated PC value is then used to access the instruction memory, allowing the corresponding instruction to be fetched. Once retrieved, the instruction is stored in the IF-ID pipeline register, which holds it temporarily for the next decode

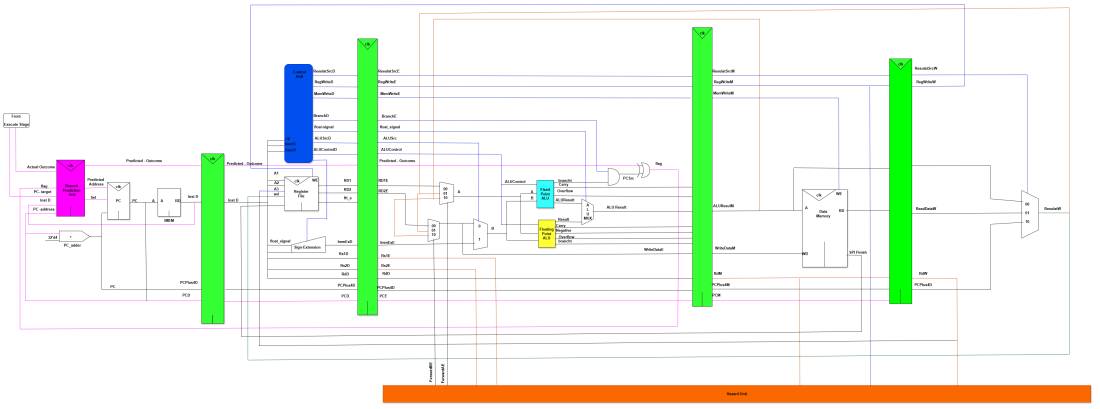

Figure 2.1: Architecture of the Proposed RISC V Core. Readers are advised to zoom in to observe finer details.

stage. Instruction memory is directly mapped to the PC, ensuring that the correct instruction is consistently accessed in every cycle.
To enhance the efficiency of this stage, a specialized module known as the Biomodule Branch Protection Unit is incorporated within the Fetch Unit. This unit is designed to minimize stalls associated with branch instructions. Typically, whether a branch is taken or not is determined only during the execution stage, resulting in up to two stall cycles for regular branch operations. The Branch Protection Unit mitigates these delays using a Finite State Machine (FSM)-based logic. Each FSM operates as a 4-state transition predictor, following the standard two-bit branch prediction scheme. The four states include: strongly not taken (00), weakly not taken (01), weakly taken (10), and strongly taken (11). These states transition based on the actual outcomes of previous branches, dynamically updating predictions. For instance, if a branch is frequently taken, the FSM remains or moves toward the strongly taken state; otherwise, it transitions toward the strongly not taken state. This predictive mechanism allows the fetch stage to make early decisions on branch outcomes, significantly reducing unnecessary stalls and improving instruction throughput. The integration of FSM-based prediction logic ensures a more intelligent and adaptive instruction flow, contributing to the overall performance and responsiveness of the processor pipeline. The efficiency of the bimodal branch predictor which is modeled using Markov Chains, denoted as $\eta(p)$, is defined as the correct prediction rate, i.e.,

$$\eta(p) = \frac{1 - 2p + 2p^2}{-4p^4 + 8p^3 - 6p^2 + 2p + 1} \tag{2.1}$$

The expression quantifies predictor accuracy as a function of branch bias p, aiding architectural optimization under varying control-flow behaviors. The worst-case scenario occurs when $p = 0.5$, resulting in an efficiency of approximately 40%. The best-case scenario occurs when $p = 0$ or $p = 1$ (or close to either), where the efficiency reaches 100%.

2.1.2 Decode Stage

The Decode Unit is responsible for interpreting the fetched instruction and generating the necessary control signals and data for execution. It comprises several key components. The Sign Extension block extends immediate values from instruction formats like I-type and S-type (12-bit to 32-bit) and J-type (26-bit to 32-bit), ensuring compatibility with the processor's 32-bit data path. The Control Unit, which is responsible for generating control signals is divided into two parts: the Main Decoder, which interprets the opcode to determine actions like register/memory write, branching, and ALU or memory operations, and the ALU Decoder, which specifies the exact ALU operation (e.g., addition, subtraction, AND, OR). The Register File reads data from the source registers (RS1, RS2) used in ALU operations and stores the result during the Write Back stage. Lastly, the ID/EX Register serves as a pipeline buffer between the Decode and Execute stages, forwarding control signals, register values, and the sign-extended immediate value to ensure stable and synchronized execution in the next stage.

2.1.3 Execute Unit

The execution unit forms a vital stage of the processor pipeline and contains the Arithmetic Logic Unit (ALU), which handles all arithmetic, logical, and branch computations. The decode unit generates specific control signals based on the instruction, and these signals guide the ALU to perform the intended operation. For example, if the decode stage outputs the control signal for addition, then the ALU executes only the addition operation; the same logic applies for subtraction, multiplication, and others.

Table 2.1: Supported Instructions in the RISC-V Processor

Category	**Instruction Type**	**Instructions**
Integer Arithmetic	R-type	ADD, SUB, MUL, DIV
	I-type	ADDI
Memory Operations	I-type	LW
	S-type	SW
Branching	B-type	BNE, BEQ, BLT, BGE
Shifting	R-type	SLL, SRL, SRA
Comparison	R-type	SLT, SLTU
Bitwise Operations	R-type	AND, OR, XOR
Floating Point Load/Store	I-type / S-type	FLW, FSW
Floating Point Arithmetic	R-type	FADD.S, FSUB.S, FMUL.S, FDIV.S

Within the execution cycle, multiplexers (MUXes) determine whether to select the immediate value or the RS2 register value as the second operand for the ALU. This selection enables support for both immediate and register-based instructions. The flags which are essential for getting the correct PC address for the branch prediction unit are calculated in the execute stage.

2.1.4 Floating Point Module

The floating point module designed in this work is a modular and IEEE-754 single-precision compliant arithmetic logic unit (ALU) that performs fundamental floating-point operations such as addition, subtraction, multiplication, and division. A key advantage of this module lies in its implementation of division using the Newton-Raphson iterative method, which accelerates the computation of reciprocals and enhances overall efficiency compared to traditional non-iterative techniques.

Furthermore, the design ensures correct rounding behavior and precision as per the IEEE-754 standard, including support for special cases such as NaN (Not-a-Number), infinities, and subnormals. Compared to conventional monolithic designs, this modular architecture offers better maintainability, scalability, and adaptability for integration into larger floating-point processing pipelines or custom processors. The RTL schematic of the floating point ALU unit is given the figure 3. Overall, the module provides a reliable and reusable foundation for high-performance floating-point computation in hardware design.

2.1.5 Memory Access and Writeback

The Memory Access and Writeback stages are vital for managing data flow between the processor and memory. In the Memory Access (MEM) stage, the ALU result from the Execute stage is used either as a memory address (for load/store instructions) or forwarded as a final computation result. Load instructions (e.g., LW) fetch data from memory, while store instructions (e.g., SW) write data—decoded earlier—from the register file into memory. This stage also interacts with the Hazard Unit to handle potential hazards, especially those caused by load-use dependencies. The Writeback (WB) stage is the final stage, responsible for writing results to the register file. A multiplexer selects between the memory-loaded data (for load instructions) and the ALU result (for arithmetic operations) to be written back. For store instructions, memory writes are already completed in the MEM stage. Additionally, the Writeback stage provides updated results to the Hazard Unit to support data forwarding and resolve instruction dependencies as illustrated in Fig 1.

2.1.6 Hazard Management System

To resolve data hazards arising from read-after-write dependencies in a pipelined processor, a hazard detection unit dynamically forwards the most recent data to the execution stage. If a source register in the execution stage matches the destination register of an instruction in the memory or writeback stage, and the corresponding instruction intends to write to the register file, the updated value is forwarded from the respective later stage. This ensures that the execution stage receives the correct operand without stalling the pipeline. The following diagram illustrates a specific scenario where such data hazards occur and are resolved through operand forwarding. One of the worst-case scenario, shown in figure 4, occurs when both source registers in the second instruction depend on the destination register of the first. Furthermore, the third instruction has data dependencies on both the first and second instructions.

2.1.7 SPI Communication Module

The SPI module is memory-mapped to address `0x01` in the data memory, enabling the processor to initiate a transmission by writing to this location. The design focuses on master-side data transmission using SPI Mode 0 (CPOL = 0, CPHA = 0). A Finite State Machine (FSM) with five distinct states—`IDLE`, `LOAD`, `TRANSMIT`, `DONE`, and `RESET`—controls the data flow. In the `IDLE` state, the module waits for a valid write signal from the processor. Upon detection, it moves to the `LOAD` state, where the 8-bit data is latched into a shift register and the slave select (SS) line is pulled low. The

`TRANSMIT` state serially shifts data out through the MOSI line on every rising edge of the SPI clock, which is internally divided from the system clock. A 3-bit counter tracks the number of bits sent. After all 8 bits are transmitted, the FSM transitions to the `DONE` state, where the SS line is deasserted, and a finish flag is raised to signal transmission completion. The processor detects this and writes the value `1` to register `x30`, indicating success. Finally, the FSM enters the `RESET` state to clear internal signals and return to `IDLE` for the next operation. The FSM-based approach ensures synchronized, bit-accurate transmission and easy interfacing within a pipelined RISC-V system.

Table 2.2: SPI Timing and Data Rate Parameters

Parameter	**Value**
Processor Clock Frequency (f_{clock})	48 MHz
Clock Cycles per SPI Bit (clk_per_bit)	2
SPI Clock Frequency (f_{SPI})	12 MHz
Time per SPI Clock Cycle (T_{SPI})	83.33 ns
Time per Byte (8 bits)	666.66 ns
Data Rate	12 Mbps (1.5 MBps)

2.1.8 FFT Co-Processor

In the designed RISC-V SoC, an FFT coprocessor is integrated to efficiently execute 8-point Decimation in Time (DIT) Fast Fourier Transform (FFT) operations. The computation follows a three-stage pipeline, where each stage handles specific butterfly operations. Stage 0 processes 2-point butterflies, Stage 1 performs 4-point FFTs, and Stage 2 completes the full 8-point transformation. Each stage includes dedicated hardwired butterfly units that operate sequentially on the input data. To avoid data hazards and maintain orderly computation, each stage only proceeds after the previous stage has completed. A Finite State Machine (FSM) governs the transitions between stages, and no data moves forward unless the receiving stage is ready, effectively managing backpressure and maintaining correct sequencing.

The main processor and FFT coprocessor execute in parallel while accessing a shared memory space. Memory-mapped architecture enables both units to interact with the same physical memory. To manage access without conflict, an arbitration mechanism assigns memory locks—one indicating the memory is occupied by the CPU and the other indicating it is in use by the FFT coprocessor. Only one unit can access memory at any given moment.

When the processor issues a memory instruction, such as load or store, it checks whether the memory is currently in use by the FFT coprocessor. If so, the processor halts in the decode stage and holds the program counter to avoid incorrect instruction execution. Once the FFT coprocessor releases the memory, the processor locks it for its own use, performs the memory operation, and then releases it, allowing further access by the FFT coprocessor.

Similarly, before accessing memory, the FFT coprocessor checks whether the processor has completed its memory transaction. If the memory is still locked by the processor, the coprocessor waits. Once the processor releases control, the coprocessor locks the memory for its own use. The coprocessor remains idle until it receives a trigger from the processor indicating that an FFT operation should begin. Once active, it reads input

data from memory, processes it across the three pipeline stages, and prepares the result. If the required memory locations for writing results are not yet ready, the coprocessor remains in a waiting state. Once those locations are available, the FFT results are written back to memory, and memory control is relinquished. At that point, the processor is notified that the FFT operation has completed, and memory is again available for general use.

Coordinated use of memory locks and readiness flags ensures that the processor and coprocessor function in parallel without interfering with each other. Arbitration logic prevents simultaneous memory access, ensuring data consistency and system stability. The decode stage in the processor plays a crucial role in reducing unnecessary stalls. If a memory instruction is encountered while the coprocessor holds memory access, the processor temporarily stalls until the coprocessor releases it. Likewise, the coprocessor waits until the processor completes its transaction before initiating access. This controlled sequencing avoids contention and minimizes processing delays.

The architecture combines pipelined FFT computation with synchronized memory arbitration to achieve efficient parallelism. Processor and coprocessor operate independently, yet remain tightly coordinated when interacting with shared memory. This balance of concurrency and control improves overall system performance, making the design highly suitable for real-time embedded systems that demand reliable and concurrent processing with minimal latency. The design achieves high parallel efficiency by combining FSM-based stage control with memory arbitration logic, enabling concurrent processing while safeguarding data integrity. By coordinating access privileges and managing pipeline dependencies, the architecture supports simultaneous execution of processor and coprocessor tasks, enhancing throughput and ensuring reliable operation for memory-intensive embedded applications. The micro-architecture of the FFT co-processor in figure 5.

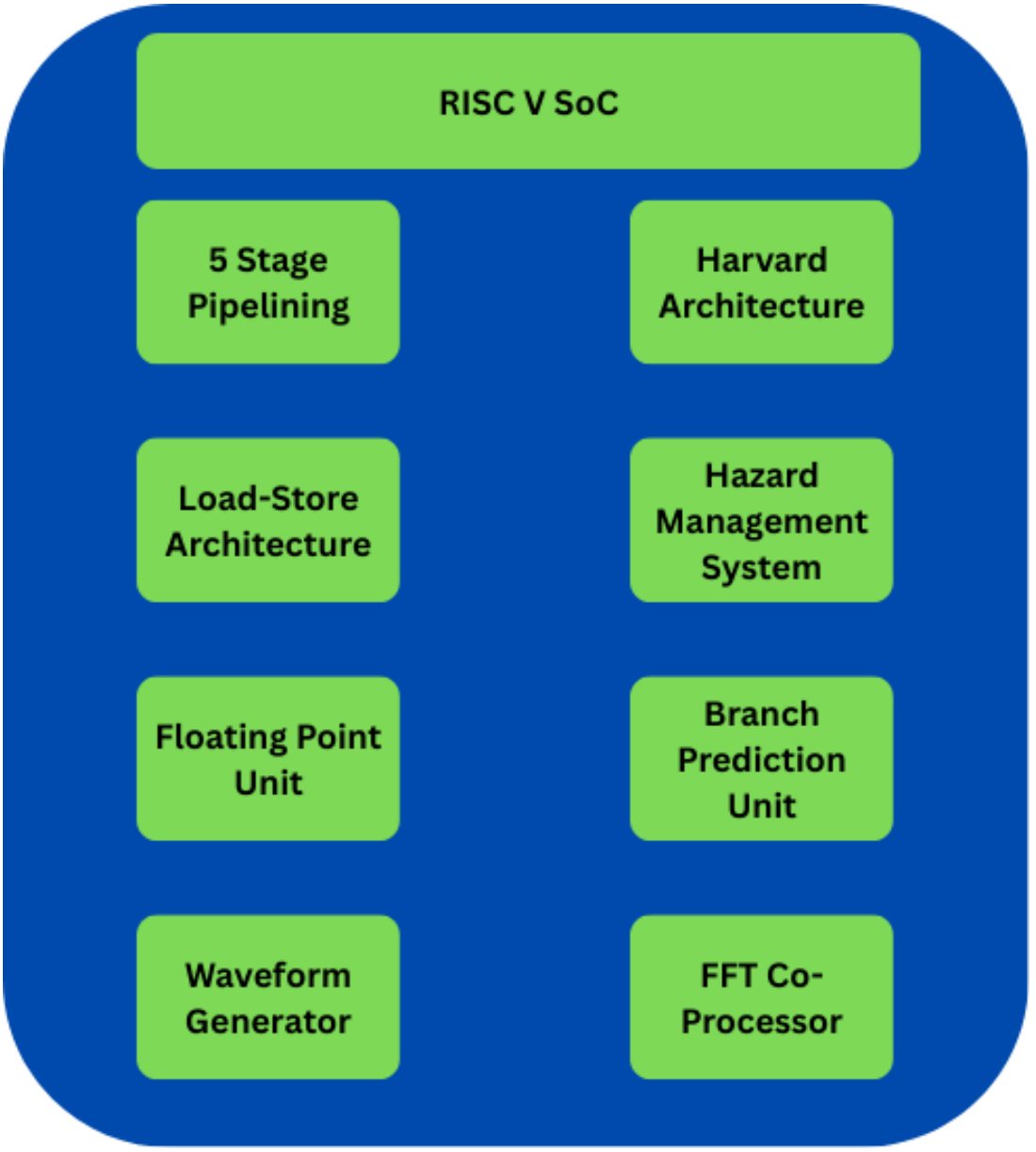

Figure 2.2: Block Diagram of the SoC

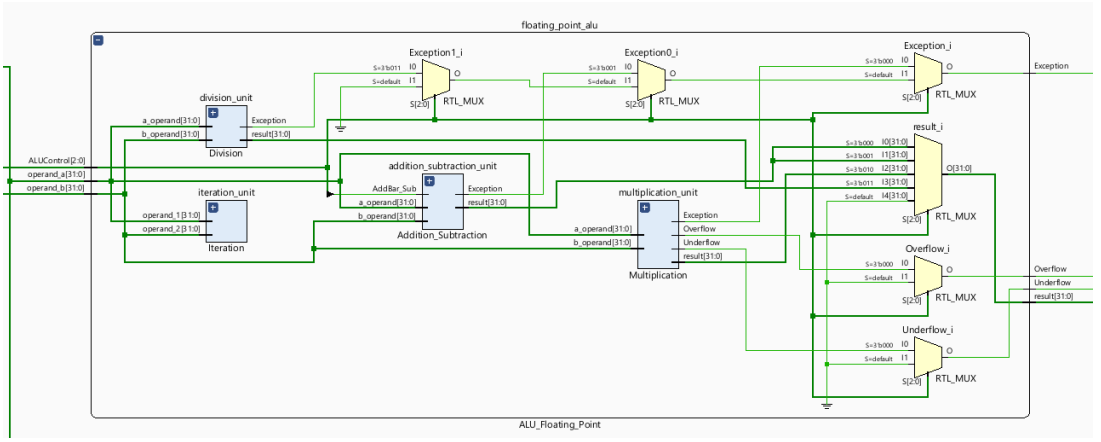

Figure 2.3: RTL Schematic of the Floating Point ALU

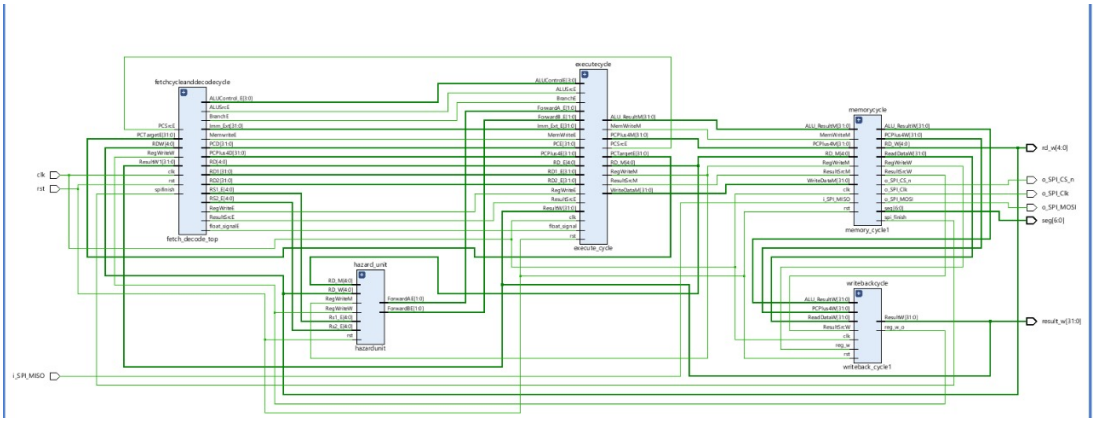

Figure 2.4: Top RTL schematic of the processor

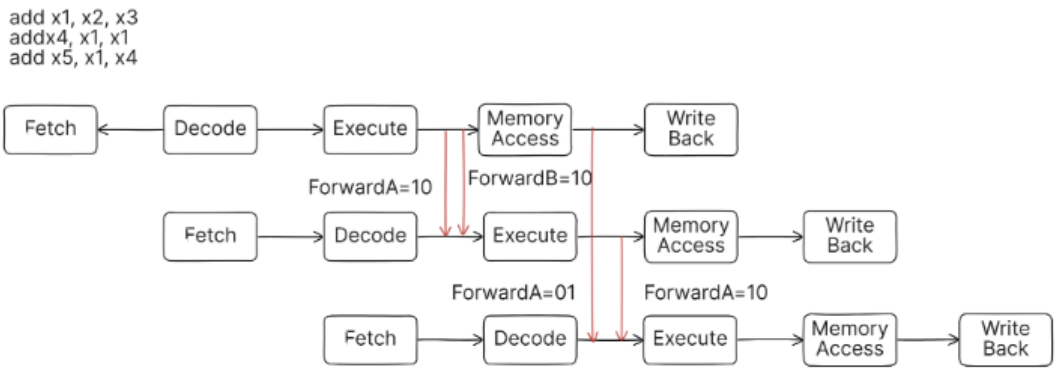

Figure 2.5: Worst Case Scenario of Hazard Unit

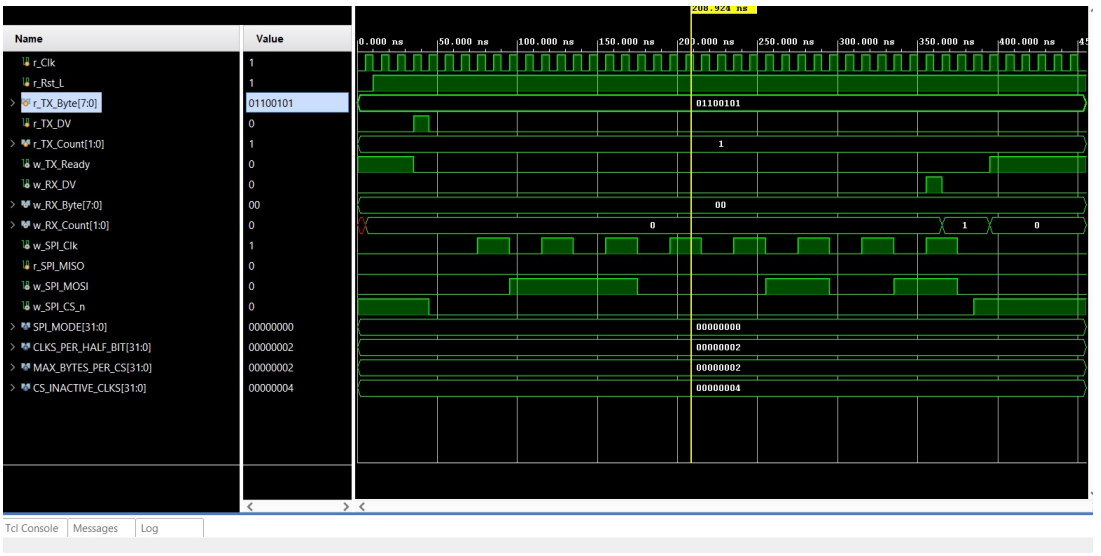

Figure 2.6: SPI communication with a slave

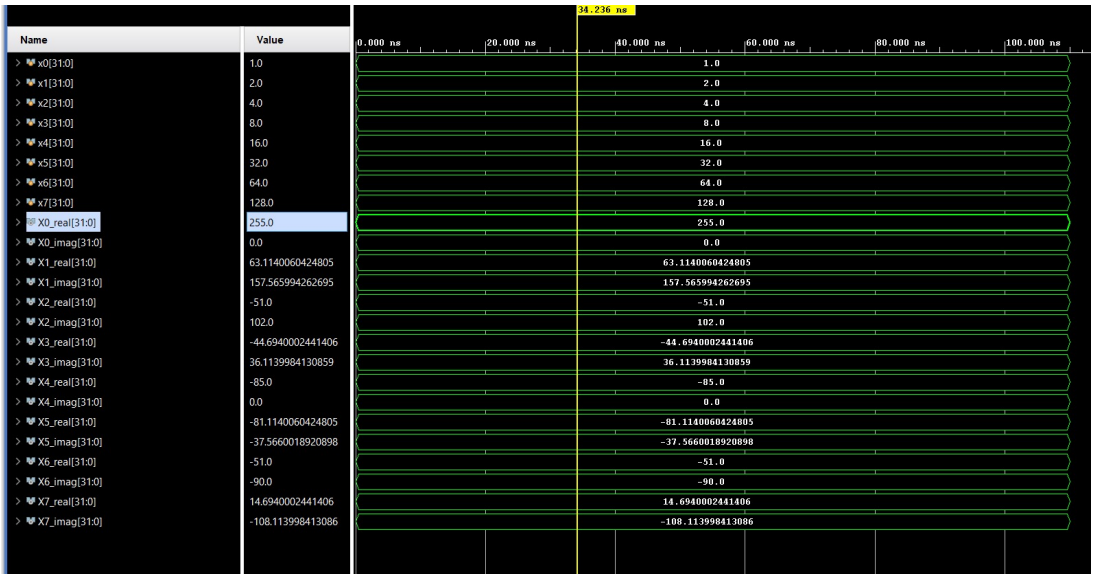

Figure 2.7: Simulation of 8 pt dit fft

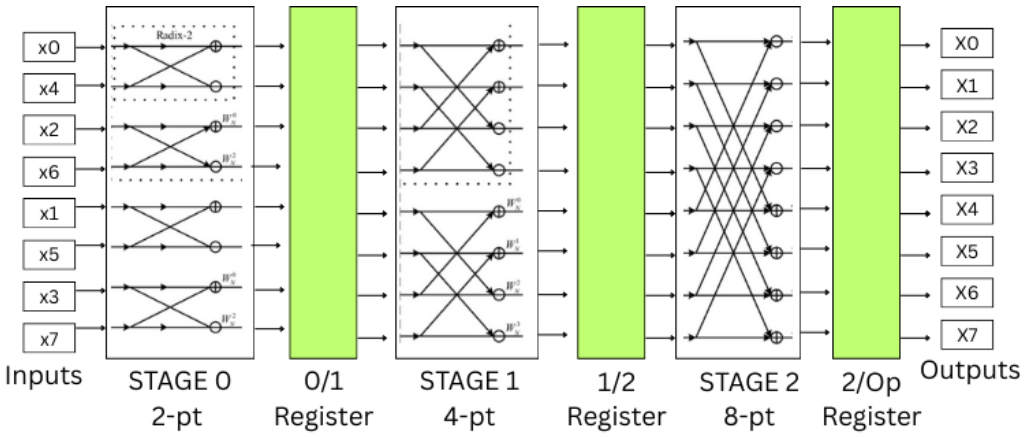

Figure 2.8: Micro-architecture of the FFT Unit

Chapter 3

Prototyping / Results

Figure 6 shows the computation of circle area using $A = \pi r^2$, where $r = 2.3$ (stored in `Register 7`) and $\pi = 3.142$ (in `Register 6`). The value r^2 is stored in `Register 9`, and the final result πr^2 in `Register 10`.
SPI communication protocol usage is demonstrated in Figure 7, where the character 'A' is displayed on the seven-segment module from the processor. Up counters and down counters have also been designed.

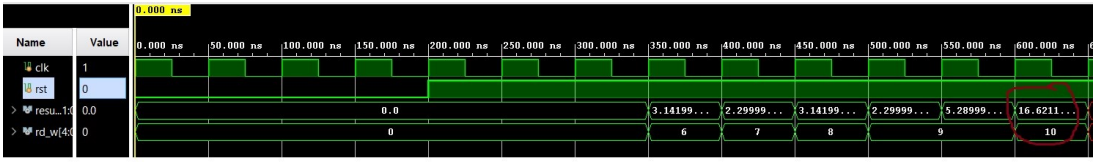

Figure 3.1: Simulation of Area of Circle

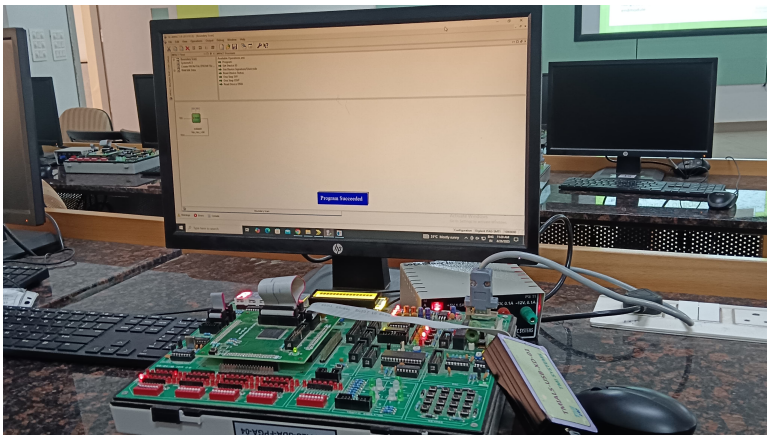

Figure 3.2: Experimental setup showing communication with a slave; 'A' displayed on the seven-segment display via the processor

3.0.1 Maximum Clock Frequency

The maximum frequency (F_{max}) is calculated using the formula:

$$F_{max} = max(\frac{1000}{T_i - WNS}) \tag{3.1}$$

The maximum frequency of the system is 48 MHz without the floating-point division module. For a lightweight RISC-V model with complex computational units such as the floating-point unit (which handles operations like addition, subtraction, and multiplication), a maximum frequency of 48 MHz is quite reasonable. However, when the floating-point division module is included, the maximum frequency drops to around 7 MHz. This reduction is due to the increased resource usage and computational complexity introduced by the division operation.

3.0.2 Power Consumption Report

Table 3.1 presents the estimated power consumption of the synthesized RISC-V processor, obtained using Vivado's power analysis tool. The processor consumes a total of **108 mW**, with a majority of the power being static.

Table 3.1: Power Consumption Report of the Synthesized Processor

Parameter	**Value**
Total On-Chip Power	0.108 W
Device Static Power	0.091 W (84%)
Dynamic Power	0.017 W (16%)
– Clocks	0.001 W (8%)
– Signals	0.006 W (37%)
– Logic	0.008 W (45%)
– DSP	0.001 W (6%)
– I/O	0.001 W (4%)
Hierarchical Power	0.017 W
Junction Temperature	25.5 °C
Thermal Margin	74.5 °C (15.9 W)
Ambient Temperature	25.0 °C
Effective θJA	4.6 °C/W

Key Observations:

- The processor exhibits low power consumption, making it suitable for energy-constrained and embedded applications.
- Static power accounts for 84% of the total power, indicating the dominance of leakage current, which is typical in FPGA-based implementations.
- Logic contributes the highest share of dynamic power (45%), reflecting core computational activity.
- Signals consume 37% of dynamic power, suggesting moderate interconnect usage and switching.
- The junction temperature remains low at 25.5 °C, well within safe thermal margins.

3.0.3 Resource Utilization

The FPGA resource utilization of the synthesized 32-bit pipelined RISC-V processor was analyzed post-synthesis using Xilinx Vivado on the Artix-7 (xc7a100t) FPGA.
It is important to note that the FFT and SPI modules are memory-mapped peripherals that operate independently of the main RISC-V pipeline. These are included in the total resource utilization for completeness but are not considered in core-specific evaluations. All optimization, performance, and area analyses for the RISC-V processor exclude these peripheral modules.

Name	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	DSPs	Bonded IOB	BUFGCTRL
TOP MODULE	26281	6080	689	118	37	34	4
Fetch Cycle	160	134	3	0	0	0	0
Decode Cycle	1098	1117	192	64	0	0	0
executecycle	8584	106	224	27	21	0	0
memorycycle	16461	4723	270	27	16	0	0

Figure 3.3: FPGA Resource Utilisation of RISC-V Processor Modules

The total LUT usage of the top module without including the FFT and SPI models is 5799 LUTs

Key Observations

- The execute-cycle module accounts for a significant share of LUT usage, indicating high arithmetic and control logic complexity.
- The Floating point ALU alone consumes 18 DSPs, highlighting its dependence on dedicated arithmetic resources.
- The memory-cycle is a major consumer of logic because the FFT unit and SPI module are mapped into it.
- The register file utilizes a high number of flip-flops (992), aligning with its function as a central storage element.
- The Branch Prediction Unit is resource-light, making it an efficient addition to the pipeline for performance gain.
- The SPI module is extremely lightweight and has negligible effect on overall resource utilization.

Chapter 4

Market Opportunity

4.0.1 Embedded Systems

Embedded systems are integral to modern electronics, with growing demand across industries. According to Fortune Business Insights, the global embedded systems market was valued at **USD 100.04 billion in 2023** and is projected to reach **USD 161.86 billion by 2030**, growing at a CAGR of **7.1%**. This growth is driven by the need for efficient, low-power processors featuring components like FPUs and specialized co-processors — especially in applications requiring real-time processing and signal analysis.
Our processor directly addresses these needs with:

- A **5-stage pipeline** for efficient, parallel instruction execution.
- **Hazard management** to ensure smooth and reliable performance under tight timing constraints.
- A **Harvard architecture** for fast, simultaneous access to instruction and data memory.
- An integrated **SPI module** for seamless communication with peripheral devices.

These features make it ideal for high-growth applications, including:

- **Drones, robotics, and smart surveillance**
- **Wearables and fitness trackers**
- **Smart home controllers**
- **Industrial automation and IIoT systems**

Our architecture enables low-latency, real-time execution for AI/ML inference, control tasks, and signal processing — aligning perfectly with current market demands.

4.0.2 Industrial IoT (IIoT) and Automation

The Industrial Internet of Things (IIoT) is undergoing rapid transformation. As reported by Grand View Research, the global IIoT market reached **USD 483.16 billion in 2024** and is forecasted to grow at a CAGR of **23.3%** from 2025 to 2030 .

This expansion demands robust, reliable processors capable of handling automation, data exchange, and control in industrial environments. The proliferation of IoT devices necessitates processors that are both powerful and energy-efficient. The number of connected IoT devices is projected to reach 40 billion by 2030

Relevant Features

- **SPI module** for seamless sensor and actuator interfacing
- **Hazard-resilient pipeline** to maintain execution integrity
- **Real-time execution** for deterministic, low-latency response

Use Cases

- Programmable Logic Controllers (PLCs), smart factory machinery
- Environmental and vibration monitoring sensors
- Motor and actuator control systems

Why Our Processor Fits

By combining robust **SPI connectivity** with a **deterministic, hazard-managed pipeline**, our processor ensures **reliable, real-time performance** in mission-critical industrial applications.

4.0.3 Digital Signal Processing (DSP) Applications

Relevant Features

- **Dedicated FFT Co-Processor** for high-speed spectral analysis
- **Floating Point Unit (FPU)** for precision in signal computation
- **Pipeline optimized for numerical throughput** to handle intensive DSP workloads

Use Cases

- Audio analysis and voice recognition
- Vibration and frequency diagnostics
- Radar, sonar, and wireless communications (SDR, LoRa, etc.)

Market Size

The global DSP market is expected to reach **USD 15.22 billion by 2030**, growing at a CAGR of **7.7%**, according to IndustryArc. This surge is driven by the rising demand for real-time data processing in applications spanning audio, telecom, and biomedical domains.

Why Our Processor Fits

While most low-end microcontrollers rely on software-based FFT processing, our design features a **hardware-accelerated FFT co-processor**, drastically improving speed and efficiency. Coupled with an **integrated FPU**, this enables **real-time, high-performance signal analysis at the edge**, making it ideal for modern DSP workloads.

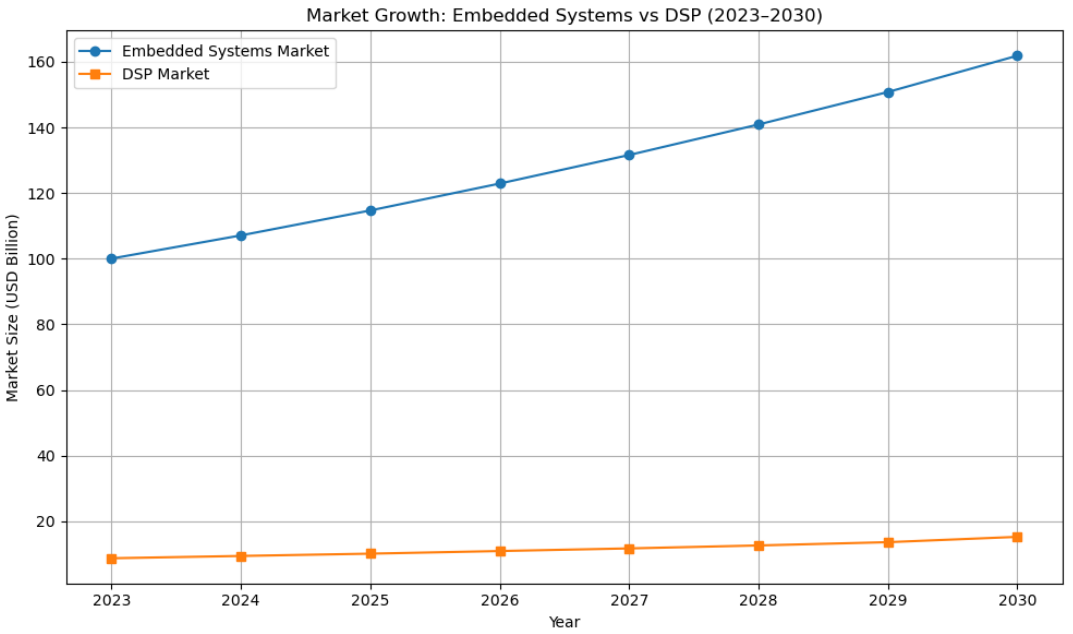

Figure 4.1: Market Growth: Embedded Systems vs DSP (2023–2030)

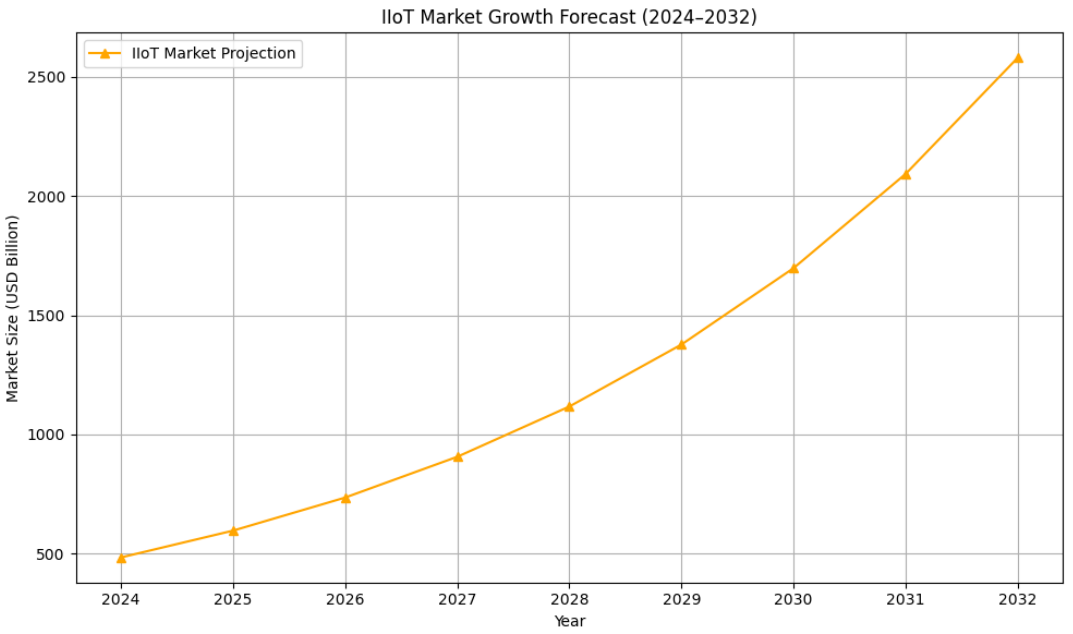

Figure 4.2: IIoT Market Growth Forecast (2024–2032)

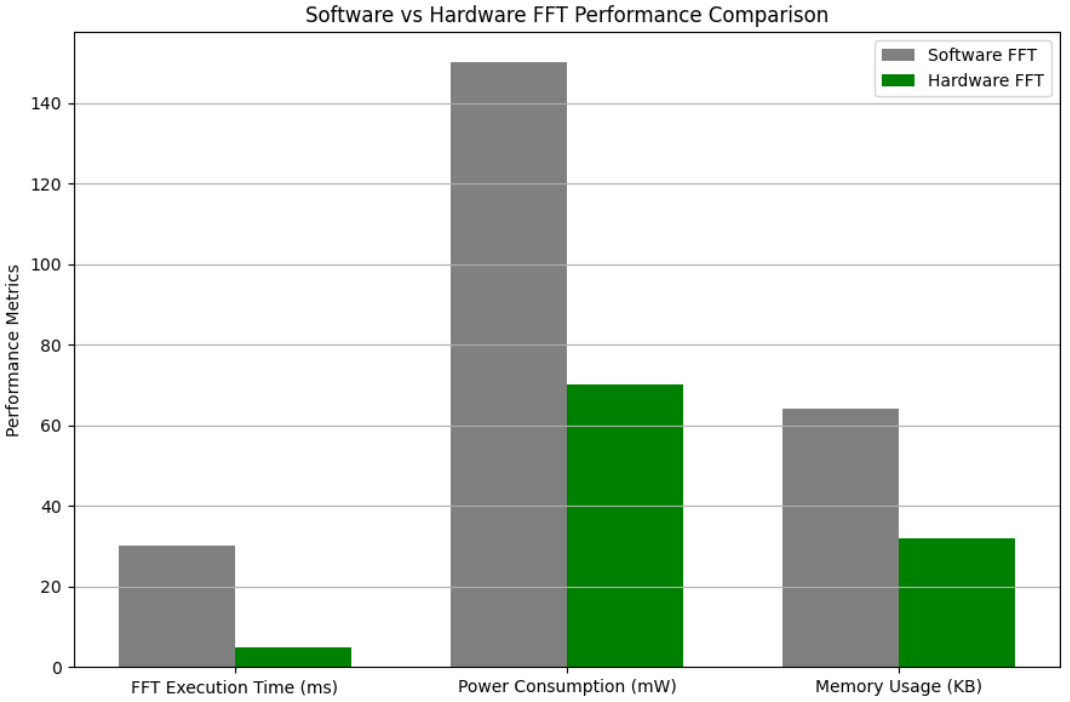

Figure 4.3: Software vs Hardware FFT Performance Comparison

Chapter 5

Competitive landscape

5.0.1 Overview

Aurora-V, our custom RISC-V-based processor, offers a unique combination of features that are rarely available together in current open-source or commercial embedded cores. This section compares Aurora-V with other popular processors in the domain.

5.0.2 Feature Comparison

Table 5.1: Core Overview of Compared Processors

Core / Processor	**Source**	**Architecture Type**
PicoRV32	Open-source	RISC-V
SiFive E21 Coreplex	Commercial	RISC-V
ARM Cortex-M4	Commercial	ARMv7-M
ESP32	Commercial	Xtensa-based
Aurora-V (Proposed)	Open/Custom	RISC-V

Table 5.2: Feature Comparison with Competitors

Feature	**Aurora-V**	**PicoRV32**	**SiFive E21**	**Cortex-M4**	**ESP32**
RISC-V ISA				×	×
5-Stage Pipeline		×	Partial		
Hazard Management		×			
Floating Point Unit		×	×		
FFT Co-processor		×	×	×	×
Harvard Architecture		×			
SPI Module					
Customizability	High	High	Medium	Low	Low
Open Source			×	×	×

5.0.3 Market Positioning

Aurora-V stands out by providing a balanced mix of high-performance execution and architectural flexibility. Figure below shows how Aurora-V compares to popular alternatives in terms of performance and customizability.

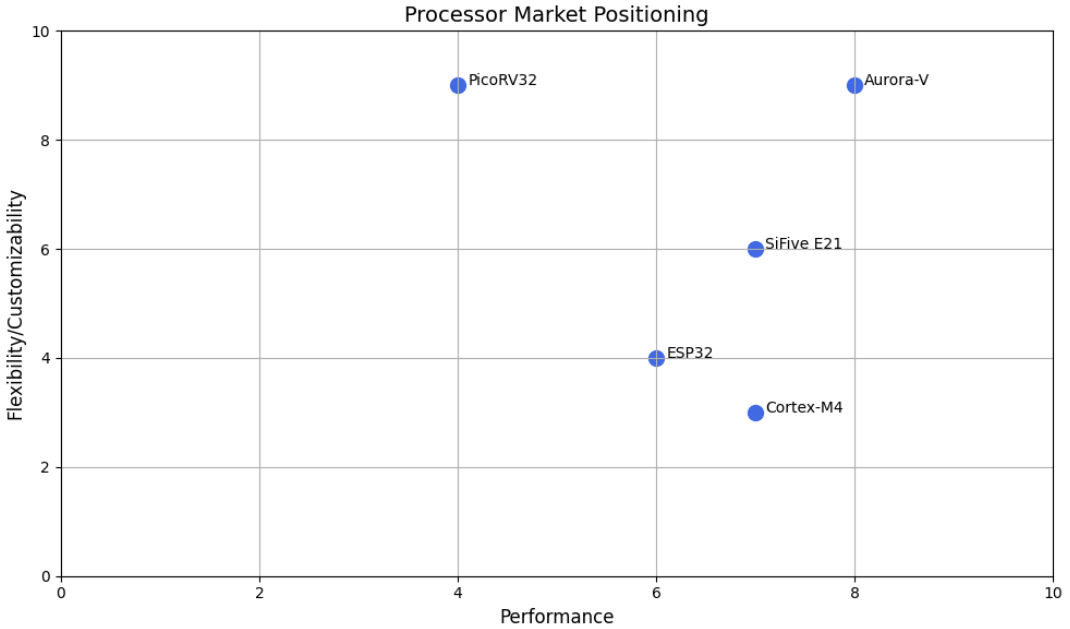

Figure 5.1: Market Positioning: Flexibility vs. Performance

Chapter 6

Budget Requirements

The table below compares the typical expected budget for a processor development project with the actual cost of the Aurora-V project, made possible by leveraging academic resources and industry partnerships.

Table 6.1: Expected Budget for Typical Processor Project

Category	**Description**	**Expected Cost (INR)**
Development Tools	FPGA board (e.g., Arty A7), programming cables	60,000
EDA Tools	Commercial EDA tools (synthesis, simulation, layout)	25,000
Fabrication & Prototyping	MPW run or custom PCB fabrication	40,000
Testing Equipment	Logic analyzers, oscilloscopes, power meters	30,000
Documentation & IP Filing	Technical documentation, copyrights/IP	10,000
Travel & Demonstration	Travel, logistics, demo setup at competition	15,000
Contingency (10%)	Unforeseen expenses buffer	18,000
Total Estimated Budget		**198,000**

Table 6.2: Actual Budget for Aurora-V Processor Development

Category	**Cost Optimization Notes**	**Aurora-V Cost (INR)**
Development Tools	Arty A7 available via lab / sponsorship	30,000
EDA Tools	Free Cadence Genuity and Xilinx tools	0
Fabrication & Prototyping	GlobalFoundries MPW slot under tie-up	0
Testing Equipment	Available in university laboratory	0
Documentation & IP Filing	Self-written documents, minimal legal filing	5,000
Travel & Demonstration	Partially covered by internal funds	10,000
Contingency	Minimal buffer for miscellaneous costs	5,000
Total Budget (Aurora-V)		**50,000**

Chapter 7

Resource Mobilisation Plan

To successfully develop the Aurora-V processor, our team has strategically mobilized human expertise, hardware, software, and institutional support. The plan below outlines the various resources involved.

7.0.1 Human Resources

Table 7.1: Human Resource Allocation

Role	**Responsibility**	**Source**
Lead Architects	Microarchitecture design, pipeline structure, hazard logic, floating-point support	Project Team
RTL Designers	Verilog coding for core, SPI, FPU, FFT, and hazard management systems	Project Team
FPGA Integration Engineers	Synthesis, timing closure, board-level implementation on Arty A7	Project Team
Documentation Leads	Reports, diagrams, pitch deck, research paper	Project Team
Faculty Mentor	Technical supervision, academic evaluation, strategic inputs	Dr. Saroja Siddamal

7.0.2 Hardware and Tools

Table 7.2: Hardware Resources

Item	**Purpose**	**Source**
Arty A7 FPGA Board	Real-time testing and implementation of Aurora-V on hardware	Institute Lab
Logic Analyzer, Oscilloscope	Debugging SPI communication, timing, and FFT outputs	University Lab
Personal Laptops	RTL design, testing, documentation	Project Team
Workstations	High-performance synthesis and simulation workloads	Institute Lab

7.0.3 Software and IP Resources

Table 7.3: Software and IP Resource Allocation

Tool / IP	**Purpose**	**Source**
Cadence Genuity Tools	Schematic capture, RTL synthesis, simulation, and layout	Academic License (Free)
Xilinx Vivado	FPGA synthesis, bitstream generation, on-board testing	Free Web Edition
Custom FFT IP Core	Dedicated hardware FFT acceleration	Designed In-house
SPI Interface Core	Slave communication module with master	Designed In-house
GitHub Repository	Version control, open collaboration, backup	Open-source platform

7.0.4 Strategic Partnerships

Table 7.4: External and Institutional Support

Partner	**Contribution**
GlobalFoundries MPW Program	Free chip fabrication through academic collaboration
Institutional Support	Access to lab infrastructure, FPGA boards, testing equipment
Open-Source RISC-V Community	Reference implementations, open tools, verification suites

Chapter 8

Summary

Aurora-V is a custom-built, performance-oriented RISC-V processor architecture designed to serve the growing demands of embedded systems and real-time digital signal processing (DSP) applications.
It features a robust 5-stage instruction pipeline with built-in hazard detection and resolution, ensuring high instruction throughput without sacrificing correctness. The integration of a dedicated **Floating-Point Unit (FPU)** extends computational precision, while the **Harvard Architecture** enables parallel instruction and data fetch, improving overall memory efficiency. A custom-developed **FFT Co-Processor** accelerates signal-processing workloads natively on hardware. Additionally, the processor includes a reliable **SPI module** for high-speed peripheral communication.

Project Highlights

- **Modular and Scalable Design:** Built from the ground up in Verilog, Aurora-V is designed for modularity, making it extensible for future SoC integration.
- **Custom Accelerators:** The FFT co-processor is tightly coupled with the pipeline, offering deterministic latency and high performance for frequency-domain applications.
- **Hardware-Proven:** Synthesized, implemented, and tested on the Xilinx Arty A7 FPGA board to validate real-world functionality and timing correctness.
- **Toolchain Efficiency:** Leveraged industry-standard tools like Cadence Genuity and Xilinx Vivado under academic licenses, ensuring professional-grade development at zero cost.

Strategic Value

- **Cost-Effective Innovation:** Achieved advanced architectural goals within an academic environment using open-source IPs, free tools, and fabrication tie-ups.
- **Research and Education Enabler:** Aurora-V can be positioned as a reference platform for RISC-V-based curriculum, lab experiments, and future capstone projects.

- **Industry Potential:** The core design is suitable for low-power edge computing, IoT nodes, biomedical signal analysis devices, and real-time controllers in robotics and automotive systems.

Next Steps

- **Fabrication:** Submit the design to the GlobalFoundries MPW academic program for silicon realization.
- **Open-Source Contribution:** Release the RTL, verification suite, and documentation on GitHub under a permissive license to foster community collaboration.
- **Startup and Grant Opportunities:** Engage with incubators, RISC-V alliances, and hardware accelerator programs to take the design toward commercialization or deeper research.